

language. It contains not only C language (such as how to use `printf()` and `scanf()`), but also other elements in a computer system (data structure, computer architecture, assembly language, linking, operating system, network...). It covers most parts of this course. You are strongly recommended to read this tutorial.

Finally, enjoy the journey of PAs, and you will find hardware is not mysterious, so does the computer system! But remember:

The machine is always right.

Every line of untested code is always wrong.

RTFM.

Reminder

This ends PA0. And there is no 必答题 in PA0.

PA1 - 开天辟地的篇章：最简单的计算机

世界诞生的故事 - 第一章

先驱已经准备好了创造计算机世界的工具。为了迈出第一步，他们运用了一些数字电路的知识，就已经创造出了一个最小的计算机——图灵机。让我们来看看其中的奥妙。

在进行本 PA 前，请在工程目录下执行以下命令进行分支整理，否则将影响你的成绩：

```
git commit --allow-empty -am "before starting pa1"
git checkout master
git merge pa0
git checkout -b pa1
```

在开始愉快的 PA 之旅之前

PA 的目的是要实现 NEMU，一款经过简化的 x86 全系统模拟器。但什么是模拟器呢？你小时候应该玩过红白机，超级玛丽，坦克大战，魂斗罗... 它们的画面是否让你记忆犹新？(希望我们之间没有代沟...) 随着时代的发展，你已经很难在市场上看到红白机的身影了。当你正在为此感到苦恼的时候，模拟器的横空出世唤醒了你心中尘封已久的童年回忆。红白机模拟器可以为你模拟出红白机的所有功能。有了它，你就好像有了一个真正的红白机，可以玩你最喜欢的红白机游戏。这里是 jyy 移植的一个小型项目 LiteNES，PA 工程里面已经带有这个项目，你可以在如今这个红白机难以寻觅的时代，再次回味你儿时的快乐时光，这实在是太神奇了！

配置 X Server

提示：如果你的实验环境是 VMWare+Ubuntu，此项可忽略。

Docker container 中默认并不带有 GUI，为了运行 LiteNES，你需要根据主机操作系统的类型，你需要下载不同的 X Server：

- Windows 用户，安装并打开 Xming。
- Mac 用户，下载安装并打开 XQuartz。
- GNU/Linux 用户，系统中已经自带 X Server，你不需要额外下载。

然后根据主机操作系统的类型，为 SSH 打开 X11 转发功能：

- Mac 用户和 GNU/Linux 用户，在运行 ssh 时加入 -X 选项即可：

```
ssh -X -p 20022 username@127.0.0.1
```

- Windows 用户，在使用 PuTTY 登陆时，在 PuTTY Configuration 窗口左侧的目录中选择 Connection->SSH->X11，在右侧勾选 EnableX 11 forwarding，然后登陆即可。

通过带有 X11 转发功能的 SSH 登陆后，在 `nexus-am/apps/litenes` 目录下执行 `make run`，即可在弹出的新窗口中运行基于 LiteNES 的超级玛丽(具体操作请参考该目录下的 README.md)。事实上，我们在 PA 进行到中期时也需要进行图像的输出，因此你务必完成 X Server 的配置。

你被计算机强大的能力征服了,你不禁思考,这到底是怎么做到的?你学习完程序设计基础课程,但仍然找不到你想要的答案.但你可以肯定的是,红白机模拟器只是一个普通的程序,因为你还是需要像运行 **Hello World** 程序那样运行它.但同时你又觉得,红白机模拟器又不像一个普通的程序,它究竟是怎么模拟出一个红白机的世界,让红白机游戏在这个世界中运行的呢?

事实上,**NEMU** 就是在做类似的事情!它模拟了一个 **x86**(准确地说, **n86** 是 **x86** 的一个子集)的世界,你可以在这个 **x86** 世界中执行程序.换句话说,你将要**在 PA 中编写一个用来执行其它程序的程序!**为了更好地理解 **NEMU** 的功能,下面将

- 在 **GNU/Linux** 中运行 **Hello World** 程序
- 在 **GNU/Linux** 中通过红白机模拟器玩超级玛丽
- 在 **GNU/Linux** 中通过 **NEMU** 运行 **Hello World** 程序

这三种情况进行比较.

```
+-----+
| "Hello World" program |
+-----+
|      GNU/Linux      |
+-----+
|   Computer hardware  |
+-----+
```

上图展示了"在 **GNU/Linux** 中运行 **Hello World** 程序"的情况. **GNU/Linux** 操作系统直接运行在计算机硬件上,对计算机底层硬件进行了抽象,同时向上层的用户程序提供接口和服务. **Hello World** 程序输出信息的时候,需要用到操作系统提供的接口,因此 **Hello World** 程序并不是直接运行在计算机硬件上,而是运行在操作系统(在这里是 **GNU/Linux**)上.

```
+-----+
|      Super Mario      |
+-----+
| Simulated NES hardware |
+-----+
|      NES Emulator     |
+-----+
|      GNU/Linux       |
+-----+
|   Computer hardware   |
+-----+
```

上图展示了"在 **GNU/Linux** 中通过红白机模拟器玩超级玛丽"的情况. 在 **GNU/Linux** 看来,运行在其上的红白机模拟器 **NES Emulator** 和上面提到的 **Hello World** 程序一样,都只不过是一个用户程序而已.神奇的是,红白机模拟器的功能是负责模拟出一套完整的红白机硬件,让超级玛丽可以在其上运行.事实上,对于超级玛丽来说,它并不能区分自己是运行在真实的红白机硬件之上,还是运行在模拟出来的红白机硬件之上,这正是"虚拟化"的魔术.

```
+-----+
| "Hello World" program |
+-----+
| Simulated x86 hardware |
+-----+
|      NEMU             |
+-----+
|      GNU/Linux       |
+-----+
|   Computer hardware   |
+-----+
```

上图展示了"在 **GNU/Linux** 中通过 **NEMU** 执行 **Hello World** 程序"的情况. 在 **GNU/Linux** 看来,运行在其上的 **NEMU** 和上面提到的 **Hello World** 程序一样,都只不过是一个用户程序而已.但 **NEMU** 的功能是负责模拟出一套 **x86** 硬件,让程序可以在其上运行.事实上,上图只是给出了对 **NEMU** 的一个基本理解,很多细节会在后续 **PA** 中逐渐补充.为了方便叙述,我们将在 **NEMU** 中运行的程序称为"客户程序".

NEMU 是什么?

上述描述对你来说也许还有些晦涩难懂, 让我们来看一个 ATM 机的例子.

ATM 机是一个物理上存在的机器, 它的功能需要由物理电路和机械模块来支撑. 例如我们在 ATM 机上进行存款操作的时候, ATM 机都会吭哧吭哧地响, 让我们相信确实是一台真实的机器. 另一方面, 现在第三方支付平台也非常流行, 例如支付宝. 事实上, 我们可以把支付宝 APP 看成一个虚拟的 ATM 机, 在这个虚拟的 ATM 机里面, 真实 ATM 机具备的所有功能, 包括存款, 取款, 查询余额, 转账等等, 都通过支付宝 APP 这个程序来实现.

同样地, NEMU 就是一个虚拟出来的计算机系统, 物理计算机中的基本功能, 在 NEMU 中都是通过程序来实现的. 要虚拟出一个计算机系统并没有你想象中的那么困难. 我们可以把计算机看成由若干个硬件部件组成, 这些部件之间相互协助, 完成"运行程序"这件事情. 在 NEMU 中, 每一个硬件部件都由一个程序相关的数据对象来模拟, 例如变量, 数组, 结构体等; 而对这些部件的操作则通过对相应数据对象的操作来模拟. 例如 NEMU 中使用数组来模拟内存, 那么对这个数组进行读写则相当于对内存进行读写.

我们可以把实现 NEMU 的过程看成是开发一个支付宝 APP. 不同的是, 支付宝具备的是真实 ATM 机的功能, 是用来交易的; 而 NEMU 具备的是物理计算机系统的功能, 是用来执行程序. 因此我们说, NEMU 是一个用来执行其它程序的程序.

你或许还对虚拟机和模拟器这两个相似的概念感到疑惑, 毕竟它们都表示用程序的功能来实现某些东西. 虚拟机就是用程序虚拟出来的机器; 而模拟器的范围则更加广泛, 可以用程序来模拟天体运动, 大气环流, 分子碰撞等等, 然而这些模拟的对象并不是一个计算机系统. 当我们用模拟器来模拟一个计算机系统的时候, 它和虚拟机在本质上并没有太大的差异. 所以我们说 NEMU 是个 x86 模拟器, 或者说 NEMU 是个 x86 的虚拟机, 其实可以认为是同一个意思: NEMU 是用程序来实现一个计算机系统的功能, 并不是一个物理上的计算机.

初识虚拟化

假设你在 Windows 中使用 Docker 安装了一个 GNU/Linux container, 然后在 container 中完成 PA, 通过 NEMU 运行 Hello World 程序. 在这样的情况下, 尝试画出相应的层次图.

嗯, 事实上在 Windows 中运行 Docker container 的真实情况有点复杂, 有兴趣的同学可以查找学习虚拟机和 container 的区别.

NEMU 的威力会让你感到吃惊! 它不仅能够运行 Hello World 这样的小程序, 在 PA 的后期, 你将会在 NEMU 中运行仙剑奇侠传(很酷!>_<). 完成 PA 之后, 你在程序设计课上对程序的认识会被彻底颠覆, 你会觉得计算机不再是一个神秘的黑盒, 甚至你会发现创造一个属于自己的计算机不再是遥不可及!

让我们来开始这段激动人心的旅程吧! 但请不要忘记:

- 机器永远是对的
- 未测试代码永远是错的
- RTFM

开天辟地的故事

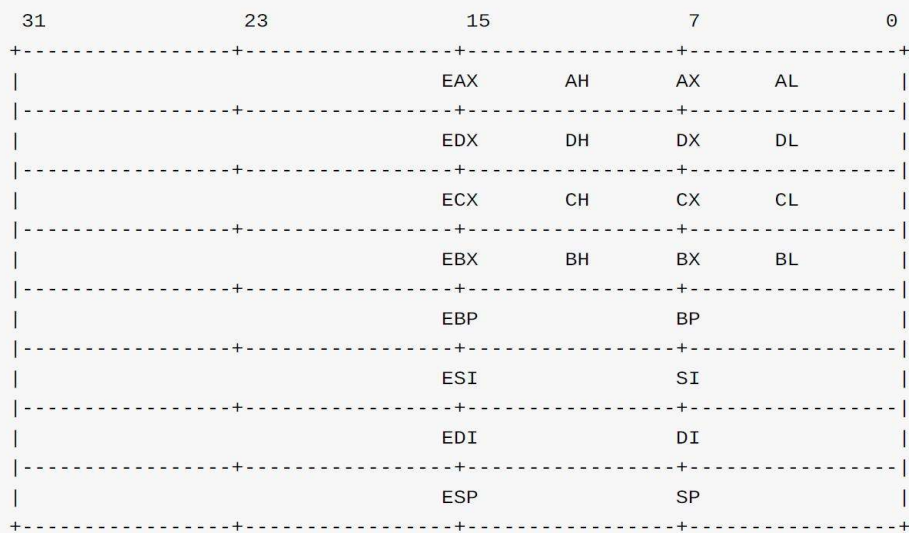
先驱希望创造一个计算机的世界, 并赋予它执行程序的使命. 让我们一起来帮助他们, 体验创世的乐趣.

大家都上过程序设计课程, 知道程序就是由代码和数据组成. 例如一个求 $1+2+\dots+100$ 的程序, 大家不费吹灰之力就可以写出一个程序来完成这件事情. 不难理解, 数据就是程序处理的对象, 代码则描述了程序希望如何处理这些数据. 先不说仙剑奇侠传这个庞然大物, 为了执行哪怕最简单的程序, 最简单的计算机又应该长什么样呢?

为了执行程序, 首先要解决的第一个问题, 就是要把程序放在哪里. 显然, 我们不希望自己创造的计算机只能执行小程序. 因此, 我们需要一个足够大容量的部件, 来放下各种各样的程序, 这个部件就是存储器. 于是, 先驱创造了存储器, 并把程序放在存储器中, 等待着 CPU 去执行.

等等, CPU 是谁? 你也许很早就听说过它了, 不过现在还是让我们来重新介绍一下它吧. CPU 是先驱最伟大的创造, 从它的中文名字"中央处理器"就看得出它被赋予了至高无上的荣耀: CPU 是负责处理数据的核心电路单元, 也就是说, 程序的执行全靠它了. 但只有存储器的计算机还是不能进行计算. 自然地, CPU 需要肩负起计算的重任, 先驱为 CPU 创造了运算器, 这样就可以对数据进行各种处理了. 如果觉得运算器太复杂, 那就先来考虑一个加法器吧.

先驱发现, 有时候程序需要对同一个数据进行连续的处理. 例如要计算 $1+2+\dots+100$, 就要对部分和 sum 进行累加, 如果每完成一次累加都需要把它写回存储器, 然后又把它从存储器中读出来继续加, 这样就太不方便了. 同时天下也没有免费的午餐, 存储器的大容量也是需要付出相应的代价的, 那就是速度慢, 这是先驱也无法违背的材料特性规律. 于是先驱为 CPU 创造了寄存器, 可以让 CPU 把正在处理中的数据暂时存放在其中. 为了兼容 x86, 我们选择了一个稍微有点复杂的寄存器结构:



其中，EAX, EDX, ECX, EBX, EBP, ESI, EDI, ESP 是 32 位寄存器；

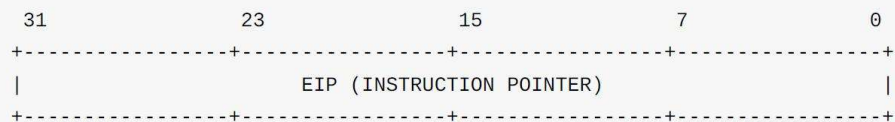
AX, DX, CX, BX, BP, SI, DI, SP 是 16 位寄存器；

AL, DL, CL, BL, AH, DH, CH, BH 是 8 位寄存器。但它们在物理上并不是相互独立的，例如 EAX 的低 16 位是 AX，而 AX 又分成 AH 和 AL。这样的结构有时候在处理不同长度的数据时能提供一些便利。

寄存器的速度很快，但容量却很小，和存储器的特性正好互补，它们之间也许会交织出新的故事呢，不过目前我们还是顺其自然吧。

为了让强大的 CPU 成为忠诚的奴仆，先驱还设计了“指令”，用来指示 CPU 对数据进行何种处理。这样，我们就可以通过指令来控制 CPU，让它做我们想做的事情了。

有了指令以后，先驱提出了一个划时代的设想：能否让程序来自动控制计算机的执行？为了实现这个设想，先驱和 CPU 作了一个简单的约定：当执行完一条指令之后，就继续执行下一条指令。但 CPU 怎么知道现在执行到哪一条指令呢？为此，先驱为 CPU 创造了一个特殊的计数器，叫“程序计数器” (Program Counter, PC)，它在 x86 中的名字叫 EIP。



从此以后，计算机就只需要做一件事情：

```
while (1) {
    从EIP指示的存储器位置取出指令；
    执行指令；
    更新EIP；
}
```

这样，我们就有了一个足够简单的计算机了。我们只要将一段指令序列放置在存储器中，然后让 PC 指向第一条指令，计算机就会自动执行这一段指令序列，永不停止。这个全自动的执行过程实在是太美妙了！事实上，开拓者图灵在 1936 年就已经提出类似的核心思想，“计算机之父”可谓名不虚传。而这个流传至今的核心思想，就是“存储程序”。为了表达对图灵的敬仰，我们也把上面这个最简单的计算机称为“图灵机” (Turing Machine, TRM)。或许你已经听说过“图灵机”这个作为计算模型时的概念，不过在这里我们只强调作为一个最简单的真实计算机需要满足哪些条件：

❖ 结构上，TRM 有存储器，有 PC，有寄存器，有加法器

❖ 工作方式上，TRM 不断地重复以下过程：从 PC 指示的存储器位置取出指令，执行指令，然后更新 PC

咦？存储器，计数器，寄存器，加法器，这些不都是数字电路课上学习过的部件吗？也许你会觉得难以置信，但先驱说，你正在面对着的那台无所不能的计算机，就是由数字电路组成的！不过，我们在程序设计课上写的程序是 C 代码。但如果计算机真的是个只能懂 0 和 1 的巨大数字电路，这个冷冰冰的电路又是如何理解凝结了人类智慧结晶的 C 代码的呢？先驱说，计算机诞生的那些年还没有 C 语言，大家都是直接编写对人类来说晦涩难懂的机器指令，那是他们所见过的最早的对电子计算机的编程方式了。后来人们发明了高级语言和编译器，能把我们写的高级语言代码进行各种处理，最后生成功能等价的，CPU 能理解的指令。CPU 执行这些指令，就相当于执行了我们写的代码。今天的计算机本质上还是“存储程序”这种天然愚钝的工作方式，是经过了无数计算机科学家们的努力，

我们今天才可以轻松地使用计算机.

RTFSC

既然 TRM 那么简单,就让我们在 NEMU 里面实现一个 TRM 吧.不过我们还是先来介绍一下框架代码.

框架代码内容众多,其中包含了很多在后续阶段中才使用的代码.随着实验进度的推进,我们会逐渐解释所有的代码.因此在阅读代码的时候,你只需要关心和当前进度相关的模块就可以了,不要纠缠于和当前进度无关的代码,否则将会给你的心灵带来不必要的恐惧。

```
ics2017
├── init.sh          # 初始化脚本
├── Makefile         # 用于工程打包提交
├── nanos-lite       # 微型操作系统内核
├── navy-apps       # 应用程序集
├── nemu            # NEMU
└── nexus-am        # 抽象计算机
```

目前我们只需要关心 NEMU 的内容. NEMU 主要由 4 个模块构成:monitor, CPU, memory, 设备. 我们已经在上一小节简单介绍了 CPU 和 memory 的功能,设备会在 PA2 中介绍,目前不必关心.monitor 位于这个虚拟计算机系统之外,主要用于监视这个虚拟计算机系统是否正确运行.monitor 从概念上并不属于一个计算机的必要组成部分,但对 NEMU 来说,它是必要的基础设施.它除了负责与 GNU/Linux 进行交互(例如读写文件)之外,还带有调试器的功能,为 NEMU 的调试提供了方便的途径.缺少 monitor 模块,对 NEMU 的调试将会变得十分困难.代码中 nemu 目录下的源文件组织如下(部分目录下的文件并未列出):

```
nemu
├── include          # 存放全局使用的头文件
│   ├── common.h    # 公用的头文件
│   ├── cpu
│   │   ├── decode.h # 译码相关
│   │   ├── exec.h   # 执行相关
│   │   ├── reg.h    # 寄存器结构体的定义
│   │   └── rtl.h     # RTL指令
│   ├── debug.h      # 一些方便调试用的宏
│   ├── device       # 设备相关
│   ├── macro.h      # 一些方便的宏定义
│   ├── memory       # 访问内存相关
│   ├── monitor
│   │   ├── expr.h   # 表达式求值相关
│   │   ├── monitor.h
│   │   └── watchpoint.h # 监视点相关
│   └── nemu.h
├── Makefile         # 指示NEMU的编译和链接
├── Makefile.git     # git版本控制相关
├── runall.sh        # 一键测试脚本
└── src              # 源文件
    ├── cpu
    │   ├── decode   # 译码相关
    │   ├── exec     # 执行相关
    │   ├── intr.c   # 中断处理相关
    │   └── reg.c     # 寄存器相关
    ├── device       # 设备相关
    ├── main.c       # 你知道的...
    ├── memory
    │   └── memory.c  # 访问内存的接口函数
    ├── misc
    │   └── logo.c    # "i386"的logo
    └── monitor
        ├── cpu-exec.c # 指令执行的主循环
        ├── diff-test
        ├── debug      # 简易调试器相关
        │   ├── expr.c  # 表达式求值的实现
        │   ├── ui.c    # 用户界面相关
        │   └── watchpoint.c # 监视点的实现
        └── monitor.c
```

为了给出一份可以运行的框架代码,代码中实现了 `mov` 指令的功能,并附带一个 `mov` 指令序列的默认客户程序.另外,部分代码中会涉及一些硬件细节.在你第一次阅读代码的时候,你需要尽快掌握 NEMU 的框架,而不要纠缠于这些细节.随着 PA 的进行,你会反复回过头来探究这些细节.大致了解上述的目录树之后,你就可以开始阅读代码了.至于从哪里开始,就不用多费口舌了吧.

对 vim 的使用感到困难?

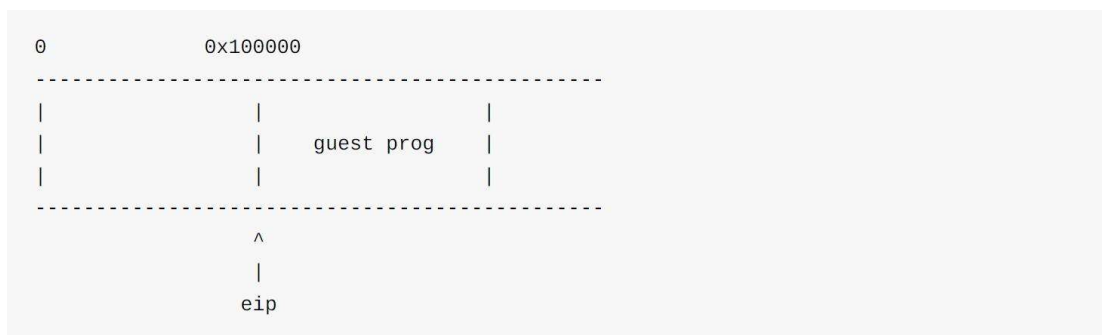
在 PA0 的强迫之下,你不得不开始学习使用 vim.如果现在你已经不再认为 vim 是个到处是 bug 的编辑器,就像简明 vim 练级攻略里面说的,你已经通过了存活阶段.接下来就是漫长的修行阶段了,每天学习一两个 vim 中的功能,累积经验值,很快你就会发现自己已经连升几级.不过最重要的还是坚持,只要你在 PA1 中坚持使用 vim,PA1 结束之后,你就会发现 vim 的熟练度已经大幅提升!你还可以搜一搜 vim 的键盘图,说不定能激发起你学习 vim 的兴趣.

NEMU 开始执行的时候,首先会调用 `init_monitor()` 函数(在 `nemu/src/monitor/monitor.c` 中定义)进行一些和 monitor 相关的初始化工作,我们对其中几项初始化工作进行一些说明.`reg_test()` 函数(在 `nemu/src/cpu/reg.c` 中定义)会生成一些随机的数据,对寄存器实现的正确性进行测试.若不正确,将会触发 `assertion fail`.

实现正确的寄存器结构体

我们在 PA0 中提到,运行 NEMU 会出现 `assertion fail` 的错误信息,这是因为框架代码并没有正确地实现用于模拟寄存器的结构体 `CPU_state`,现在你需要实现它了(结构体的定义在 `nemu/include/cpu/reg.h` 中).关于 i386 寄存器的更多细节,请查阅 i386 手册. Hint: 使用匿名 union.

然后通过调用 `load_img()` 函数(在 `nemu/src/monitor/monitor.c` 中定义)读入带有客户程序的镜像文件.我们知道内存是一种 RAM,是一种易失性的存储介质,这意味着计算机刚启动的时候,内存中的数据都是无意义的;而 BIOS 是固化在 ROM 中的,它是一种非易失性的存储介质,BIOS 中的内容不会因为断电而丢失.因此在真实的计算机系统中,计算机启动后首先会把控制权交给 BIOS,BIOS 经过一系列初始化工作之后,再从磁盘中将有意義的程序读入内存中执行.对这个过程的模拟需要了解很多超出本课程范围的细节,我们在这里做了简化,让 monitor 直接把一个有意义的客户程序镜像 `guest prog` 读入到一个固定的内存位置 `0x100000`,这个程序是运行 NEMU 的一个参数,在运行 NEMU 的命令中指定,缺省时将把上文提到的 `mov` 程序作为客户程序(参考 `load_default_img()` 函数).这时内存的布局如下:



然后调用 `restart()` 函数(在 `nemu/src/monitor/monitor.c` 中定义),它模拟了"计算机启动"的功能,进行一些和"计算机启动"相关的初始化工作,一个重要的工作就是将 `%eip` 的初值设置为刚才我们约定的内存位置 `0x100000`,这样就可以让 CPU 从我们约定的内存位置开始执行程序了.

monitor 的其它初始化工作我们会在后续实验内容中介绍,目前可以不必关心它们的细节,最后通过调用 `welcome()` 函数输出欢迎信息和 NEMU 的编译时间.monitor 的初始化工作结束后,NEMU 会进入用户界面主循环 `ui_mainloop()` (在 `nemu/src/monitor/debug/ui.c` 中定义),输出 NEMU 的命令提示符: `(nemu)`

代码已经实现了几个简单的命令,它们的功能和 GDB 是很类似的.输入 `c` 之后,NEMU 开始进入指令执行的主循环 `cpu_exec()` (在 `nemu/src/monitor/cpu-exec.c` 中定义).`cpu_exec()` 模拟了 CPU 的工作方式:不断执行指令.`exec_wrapper()` 函数(在 `nemu/src/cpu/exec/exec.c` 中定义)的功能让 CPU 执行当前 `%eip` 指向的一条指令,然后更新 `%eip`.已经执行的指令会输出到日志文件 `log.txt` 中,你可以打开 `log.txt` 来查看它们.

究竟要执行多久?

在 `cmd_c()` 函数中,调用 `cpu_exec()` 的时候传入了参数 `-1`,你知道这是什么意思吗?

执行指令的相关代码在 `nemu/src/cpu/exec` 目录下.其中一个重要的部分定义在 `nemu/src/cpu/exec/exec.c` 文件中的 `opcode_table` 数组,在这个数组中,你可以看到框架代码中都已经实现了哪些指令.其中 `EMPTY` 代表对应的指令还没有实现(也可能是 x86 中不存在该指令).在以后的 PA 中,随着你实现越来越多的指令,这个数组会逐渐被它们代替.关于指令执行的详细解释和 `exec_wrapper()` 相关的内容需要涉及很多细节,目前你不必关心,我们将会在 PA2 中进行解释.

NEMU 将不断执行指令,直到遇到以下情况之一,才会退出指令执行的循环:

- 达到要求的循环次数.
- 客户程序执行了 `nemu_trap` 指令.这是一条特殊的指令,机器码为 `0xd6`.如果你查阅 `i386` 手册,你会发现 `x86` 中并没有这条指令,它是为了在 NEMU 中让客户程序指示执行的结束而加入的.

当你看到 NEMU 输出以下内容时:`nemu: HIT GOOD TRAP at eip = 0x00100026`

说明客户程序已经成功地结束运行.退出 `cpu_exec()` 之后,NEMU 将返回到 `ui_mainloop()`,等待用户输入命令.但为了再次运行程序,你需要键入 `q` 退出 NEMU,然后重新运行.

谁来指示程序的结束?

在程序设计课上老师告诉你,当程序执行到 `main()` 函数返回处的时候,程序就退出了,你对此深信不疑.但你是否怀疑过,凭什么程序执行到 `main()` 函数的返回处就结束了?如果有人告诉你,程序设计课上老师的说法是错误的,你有办法来证明/反驳吗?如果你对此感兴趣,请在互联网上搜索相关内容.

最后我们聊聊代码中一些值得注意的地方.

◆ 三个对调试有用的宏(在 `nemu/include/debug.h` 中定义)

- ✧ `Log()` 是 `printf()` 的升级版,专门用来输出调试信息,同时还会输出使用 `Log()` 所在的源文件,行号和函数.当输出的调试信息过多的时候,可以很方便地定位到代码中的相关位置
- ✧ `Assert()` 是 `assert()` 的升级版,当测试条件为假时,在 `assertion fail` 之前可以输出一些信息
- ✧ `panic()` 用于输出信息并结束程序,相当于无条件的 `assertion fail`

代码中已经给出了使用这三个宏的例子,如果你不知道如何使用它们,RTFSC.

◆ 内存通过在 `nemu/src/memory/memory.c` 中定义的大数组 `pmem` 来模拟.在客户程序运行的过程中,总是使用 `vaddr_read()` 和 `vaddr_write()` 访问模拟的内存.`vaddr`,`paddr` 分别代表虚拟地址和物理地址.这些概念在将来会用到,但从现在开始保持接口的一致性可以在将来避免一些不必要的麻烦.

理解框架代码

你需要结合上述文字理解 NEMU 的框架代码.需要注意的是,阅读代码也是有技巧的,如果你分开阅读框架代码和上述文字,你可能会觉得阅读之后没有任何效果.因此,你需要一边阅读上述文字,一边阅读相应的框架代码.

如果你不知道"怎么才算是看懂了框架代码",你可以先尝试进行后面的任务.如果发现不知道如何下手,再回来仔细阅读这一页面.理解框架代码是一个螺旋上升的过程,不同的阶段有不同的重点.你不必因为看不懂某些细节而感到沮丧,更不要试图一次把所有代码全部看明白.讲义中的知识点很多,在实验的不同阶段对同一个知识点的理解也会有所不同.我们建议你在完成相应阶段的任务之后回过头来重新阅读一遍讲义的内容,你很可能会有不一样的收获.

事实上,TRM 的实现已经都蕴含在上述的介绍中了.

- 存储器是个在 `nemu/src/memory/memory.c` 中定义的大数组
- PC 和通用寄存器都在 `nemu/include/cpu/reg.h` 中的结构体中定义
- 加法器在...嗯,框架代码这部分的内容有点复杂,不过它并不影响我们对 TRM 的理解,我们还是在 PA2 里面再介绍它吧
- TRM 的工作方式通过 `cpu_exec()` 和 `exec_wrapper()` 体现

基础设施:简易调试器

基础设施-提高项目开发的效率

基础设施是指支撑项目开发的各种工具和手段.原则上基础设施并不属于课本上知识的范畴,但是作为一个有一定规模的项目,基础设施的好坏甚至会影响到项目的推进,这是你在程序设计课上体会不到的.

事实上,你已经体会过基础设施给你带来的便利了.我们的框架代码已经提供了 `Makefile` 来对 NEMU 进行一键编译.现在我们来假设我们没有提供一键编译的功能,你需要通过手动键入 `gcc` 命令的方式来编译源文件:假设你手动输入一条 `gcc` 命令需要 10 秒的时间(你还需要输入很多编译选项,能用 10 秒输入完已经是非常快的了),而 NEMU 工程下有 30 个源文件,为了编译出 NEMU 的可执行文件,你需要花费多少时间?然而你还需要在开发 NEMU 的过程中不断进行编译,假设你需要编译 500 次 NEMU 才能完成 PA,一学期下来,你仅仅花在键入编译命令上的时间有多少?

有的项目即使使用工具也需要花费较多时间来构建.例如硬件开发平台 `vivado` 一般需要花费半小时到一小时不等的时间来生成比特文件,也就是说,你编写完代码之后,可能需要等待一小时之后才能验证你的代码是否正确.这是因为,这个过程不像编译程序这么简单,其中需要处理很多算法上的 NPC 问题.为了生成一个还不错的比特文件,`vivado` 需要付出比 `gcc` 更大的代价来解决这些 NPC 问题.这时候基础设施的作用就更加重要了,如果能有工具可以帮助你一次进行多个方面的验证,就会帮助你节省下来无数个"一小时".

Google 内部的开发团队非常重视基础设施的建设,他们把可以让一个项目得益的工具称为 **Adder**,把可以让多个项目都得益的工具称为 **Multiplier**. 顾名思义,这些工具可以成倍提高项目开发的效率. 在学术界,不少科研工作的目标也是提高开发效率,例如自动 bug 检测和修复,自动化验证,易于开发的编程模型等等. 在 PA 中,基础设施也会体现在不同的方面,我们会在将来对其它方面进行讨论.

你将来肯定会参与比 PA 更大的项目,如何提高项目开发的效率也是一个很重要的问题. 希望在完成 PA 的过程中,你能够对基础设施有新的认识:有代码的地方,就有基础设施. 随着知识的积累,将来的你或许也会投入到这些未知的领域当中,为全世界的开发者作出自己的贡献.

简易调试器是 NEMU 中一项非常重要的基础设施. 我们知道 NEMU 是一个用来执行其它客户程序的程序,这意味着, NEMU 可以随时了解客户程序执行的所有信息. 然而这些信息对外面的调试器(例如 GDB)来说,是不容易获取的. 例如在通过 GDB 调试 NEMU 的时候,你将很难在 NEMU 中运行的客户程序中设置断点,但对于 NEMU 来说,这是一件不太困难的事情.

为了提高调试的效率,同时也作为熟悉框架代码的练习,我们需要在 monitor 中实现一个具有如下功能的简易调试器(相关部分的代码在 nemu/src/monitor/debug 目录下),如果你不清楚命令的格式和功能,请参考如下表格:

命令	格式	使用举例	说明
帮助(1)	help	help	打印命令的帮助信息
继续运行(1)	c	c	继续运行被暂停的程序
退出(1)	q	q	退出 NEMU
单步执行	si [N]	si 10	让程序单步执行 N 条指令后暂停执行,当 N 没有给出时,缺省为 1
打印程序状态	info SUBCMD	info r info w	打印寄存器状态 打印监视点信息
表达式求值	p EXPR	p \$eax + 1	求出表达式 EXPR 的值, EXPR 支持的运算请见调试中的表达式求值小节
扫描内存(2)	x N EXPR	x 10 \$esp	求出表达式 EXPR 的值,将结果作为起始内存地址,以十六进制形式输出连续的 N 个 4 字节
设置监视点	w EXPR	w *0x2000	当表达式 EXPR 的值发生变化时,暂停程序执行
删除监视点	d N	d 2	删除序号为 N 的监视点

备注:

(1) help, c, q 命令已实现

(2) 与 GDB 相比,我们在这里做了简化,更改了命令的格式

总有一天会找上门来的 bug

你需要在将来的 PA 中使用这些功能来帮助你进行 NEMU 的调试. 如果你的实现是有问题的,将来你有可能面临以下悲惨的结局:你实现了某个新功能之后,打算对它进行测试,通过扫描内存的功能来查看一段内存,发现输出并非预期结果. 你认为是刚才实现的新功能有问题,于是对它进行调试. 经过了几天几夜的调试之后,你泪流满面地发现,原来是扫描内存的功能有 bug!

如果你想避免类似的悲惨结局,你需要在实现一个功能之后对它进行充分的测试. 随着时间的推移,发现同一个 bug 所需要的代价会越来越大.

解析命令

NEMU 通过 `readline` 库与用户交互,使用 `readline()` 函数从键盘上读入命令. 与 `gets()` 相比, `readline()` 提供了"行编辑"的功能,最常用的功能就是通过上,下方向键翻阅历史记录. 事实上, shell 程序就是通过 `readline()` 读入命令的. 关于 `readline()` 的功能和返回值等信息,请查阅 `man readline`

从键盘上读入命令后, NEMU 需要解析该命令,然后执行相关的操作. 解析命令的目的是识别命令中的参数,例如在 `si 10` 的命令中识别出 `si` 和 `10`,从而得知这是一条单步执行 10 条指令的命令. 解析命令的工作是通过一系

列的字符串处理函数来完成的,例如框架代码中的 `strtok()`。`strtok()` 是 C 语言中的标准库函数,如果你从来没有使用过 `strtok()`,并且打算继续使用框架代码中的 `strtok()` 来进行命令的解析,请务必查阅 `man strtok`

另外, `cmd_help()` 函数中也给出了使用 `strtok()` 的例子。事实上,字符串处理函数有很多,键入以下内容: `man 3 strtok` 其中 `<TAB>` 代表键盘上的 TAB 键。你会看到很多以 `str` 开头的函数,其中有你应该很熟悉的 `strlen()`, `strcpy()` 等函数。你最好都先看看这些字符串处理函数的 `manual page`,了解一下它们的功能,因为你很可能会用到其中的某些函数来帮助你解析命令。当然你也可以编写你自己的字符串处理函数来解析命令。

另外一个值得推荐的字符串处理函数是 `sscanf()`,它的功能和 `scanf()` 很类似,不同的是 `sscanf()` 可以从字符串中读入格式化的内容,使用它有时候可以很方便地实现字符串的解析。如果你从来没有使用过它们,RTFM,或者到互联网上查阅相关资料。

单步执行

单步执行的功能十分简单,而且框架代码中已经给出了模拟 CPU 执行方式的函数,你只要使用相应的参数去调用它就可以了。如果你仍然不知道要怎么做,RTFSC。

打印寄存器

打印寄存器就更简单了,执行 `info r` 之后,直接用 `printf()` 输出所有寄存器的值即可。如果你从来没有使用过 `printf()`,请到互联网上搜索相关资料。如果你不知道要输出什么,你可以参考 GDB 中的输出。

扫描内存

扫描内存的实现也不难,对命令进行解析之后,先求出表达式的值。但你还没有实现表达式求值的功能,现在可以先实现一个简单的版本:规定表达式 `EXPR` 中只能是一个十六进制数,例如 `x 10 0x100000`。这样的简化可以让你暂时不必纠缠于表达式求值的细节。解析出待扫描内存的起始地址之后,你就使用循环将指定长度的内存数据通过十六进制打印出来。如果你不知道要怎么输出,同样的,你可以参考 GDB 中的输出。

实现了扫描内存的功能之后,你可以打印 `0x100000` 附近的内存,你应该会看到程序的代码,和默认镜像进行对比,看看你的实现是否正确。

实现单步执行, 打印寄存器, 扫描内存

熟悉了 NEMU 的框架之后,这些功能实现起来都很简单,同时我们对输出的格式不作硬性规定,就当是熟悉 GNU/Linux 编程的一次练习吧。

不敢下手?别怕,放手去写!编译运行就知道写得对不对。代码改挂了,就改回来呗;代码改得面目全非,还 git 呀!

温馨提示

PA1 阶段 1 到此结束。

表达式求值

给你一个表达式的字符串 `"5+4*3/2-1"`,你如何求出它的值?表达式求值是一个很经典的问题,以至于有很多方法来解决它。我们在所需知识和难度两方面做了权衡,在这里使用如下方法来解决表达式求值的问题:

- ❖ 首先识别出表达式中的单元
- ❖ 根据表达式的归纳定义进行递归求值

词法分析

"词法分析"这个词看上去很高端,说白了就是"识别出表达式中的单元"。这里的"单元"是指有独立含义的字符串,它们正式的称呼叫 `token`。具体地说,我们需要在上述表达式中识别出 `5`, `+`, `4`, `*`, `3`, `/`, `2`, `-`, `1` 这些 `token`。你可能会觉得这是一件很简单的事情,但考虑以下的表达式:

```
"0xc0100000+($eax+5)*4-*( $ebp+8) number"
```

它包含更多的功能,例如十六进制整数 (`0xc0100000`),小括号,访问寄存器 (`$eax`),指针解引用 (第二个 `*`),访问变量 (`number`)。事实上,这种复杂的表达式在调试过程中经常用到,而且你需要在空格数目不固定 (0 个或多个) 的情况下仍然能正确识别出其中的 `token`。当然你仍然可以手动进行处理 (如果你喜欢挑战性的工作的话),一种更方便快捷的做法是使用正则表达式。正则表达式可以很方便地匹配出一些复杂的 `pattern`,是程序员必须掌握的内容。如果你从来没有接触过正则表达式,请查阅相关资料。在实验中,你只需要了解正则表达式的一些基本知识就可以了 (例如元字符)。

学会使用简单的正则表达式之后,你就可以开始考虑如何利用正则表达式来识别出 token 了。我们先来处理算术表达式,即待求值表达式中只允许出现以下的 token 类型:

- 十进制整数
- +, -, *, /
- (,)
- 空格串(一个或多个空格)

首先我们需要使用正则表达式分别编写用于识别这些 token 类型的规则。在框架代码中,一条规则是由正则表达式和 token 类型组成的二元组。框架代码中已经给出了+和空格串的规则,其中空格串的 token 类型是 TK_NOTYPE,因为空格串并不参加求值过程,识别出来之后就可以将它们丢弃了;+的 token 类型是'+'。事实上 token 类型只是一个整数,只要保证不同的类型的 token 被编码成不同的整数就可以了。框架代码中还有一条用于识别双等号的规则,不过我们现在可以暂时忽略它。

这些规则会在 NEMU 初始化的时候被编译成一些用于进行 pattern 匹配的内部信息,这些内部信息是被库函数使用的,而且它们会被反复使用,但你不必关心它们如何组织。但如果正则表达式的编译不通过, NEMU 将会触发 assertion fail,此时你需要检查编写的规则是否符合正则表达式的语法。

给出一个待求值表达式,我们首先要识别出其中的 token,进行这项工作的是 make_token() 函数。make_token() 函数的工作方式十分直接,它用 position 变量来指示当前处理到的位置,并且按顺序尝试用不同的规则来匹配当前位置的字符串。当一条规则匹配成功,并且匹配出的子串正好是当前待处理串的起始位置时,我们就成功地识别出一个 token, Log() 宏会输出识别成功的信息。你需要做的是将识别出的 token 信息记录下来(一个例外是空格串),我们使用 Token 结构体来记录 token 的信息:

```
typedef struct token {
    int type;
    char str[32];
} Token;
```

其中 type 成员用于记录 token 的类型。大部分 token 只要记录类型就可以了,例如+, -, *, /, 但这对于有些 token 类型是不够的:如果我们只记录了一个十进制整数 token 的类型,在进行求值的时候我们还是不知道这个十进制整数是多少。这时我们应该将 token 相应的子串也记录下来, str 成员就是用来做这件事情的。需要注意的是, str 成员的长度是有限的,当你发现缓冲区将要溢出的时候,要进行相应的处理(思考一下,你会如何进行处理?),否则将会造成难以理解的 bug。tokens 数组用于按顺序存放已经被识别出的 token 信息, nr_token 指示已经被识别出的 token 数目。

如果尝试了所有的规则都无法在当前位置识别出 token,识别将会失败,这通常是待求值表达式并不合法造成的, make_token() 函数将返回 false,表示词法分析失败。

系统设计的黄金法则 —— KISS 法则

这里的 KISS 是 Keep It Simple, Stupid 的缩写,它的中文翻译是:不要在一开始追求绝对的完美。你已经学习过程序设计基础,这意味着你已经学会写程序了,但这并不意味着你可以顺利地 PA,因为在现实世界中,我们需要的是可以运行的 system,而不是求阶乘的小程序。NEMU 作为一个麻雀虽小,五脏俱全的小型系统,其代码量达到 3000 多行(不包括空行)。随着 PA 的进行,代码量会越来越多,各个模块之间的交互也越来越复杂,工程的维护变得越来越困难,一个很弱智的 bug 可能需要调好几天。在这种情况下,系统能跑起来才是王道,跑不起来什么都是浮云,追求面面俱到只会增加代码维护的难度。

唯一可以把 you 从 bug 的混沌中拯救出来的就是 KISS 法则,它的宗旨是从易到难,逐步推进,一次只做一件事,少做无关的事。如果你不知道这是什么意思,我们上文提到的 str 成员缓冲区溢出问题来作为例子。KISS 法则告诉你,你应该使用 assert(0), 就算不"得体"地处理上述问题,仍然不会影响表达式求值的核心功能的正确性。如果你还记得调试公理,你会发现两者之间是有联系的:调试公理第二点告诉你,未测试代码永远是错的。与其一下子写那么多"错误"的代码,倒不如使用 assert(0) 来有效帮助你减少这些"错误"。

如果把 KISS 法则放在软件工程领域来解释,它强调的就是多做单元测试:写一个函数,对它进行测试,正确之后再写下一个函数,再对它进行测试... 一种好的测试方式是使用 assertion 进行验证, reg_test() 就是这样的例子。学会使用 assertion, 对程序的测试和调试都百利而无一害。

KISS 法则不但广泛用在计算机领域,就连其它很多领域也视其为黄金法则,这里有一篇文章举出了很多的例子,我们强烈建议你阅读它,体会 KISS 法则的重要性。

实现算术表达式的词法分析

你需要完成以下的内容:

- 为算术表达式中的各种 token 类型添加规则, 你需要注意 C 语言字符串中转义字符的存在和正则表达式中元字符的功能.
- 在成功识别出 token 后, 将 token 的信息依次记录到 tokens 数组中.

递归求值

把待求值表达式中的 token 都成功识别出来之后, 接下来我们就可以进行求值了. 需要注意的是, 我们现在是在对 tokens 数组进行处理, 为了方便叙述, 我们称它为 "token 表达式". 例如待求值表达式 "4+3*(2-1)" 的 token 表达式为

```
+-----+-----+-----+-----+-----+-----+-----+-----+
| NUM | '+' | NUM | '*' | '(' | NUM | '-' | NUM | ')' |
| "4" |    | "3" |    |    | "2" |    | "1" |    |
+-----+-----+-----+-----+-----+-----+-----+-----+
```

根据表达式的归纳定义特性, 我们可以很方便地使用递归来进行求值. 首先我们给出算术表达式的归纳定义:

```
<expr> ::= <number>      # 一个数是表达式
| "(" <expr> ")"          # 在表达式两边加个括号也是表达式
| <expr> "+" <expr>       # 两个表达式相加也是表达式
| <expr> "-" <expr>       # 接下来你全懂了
| <expr> "*" <expr>
| <expr> "/" <expr>
```

上面这种表示方法就是大名鼎鼎的 BNF, 任何一本正规的程序设计语言教程都会使用 BNF 来给出这种程序设计语言的语法.

根据上述 BNF 定义, 一种解决方案已经逐渐成型了: 既然长表达式是由短表达式构成的, 我们就先对短表达式求值, 然后再对长表达式求值. 这种十分自然的解决方案就是分治法的应用, 就算你没听过这个高大上的名词, 也不难理解这种思路. 而要实现这种解决方案, 递归是你的不二选择.

为了在 token 表达式中指示一个子表达式, 我们可以使用两个整数 p 和 q 来指示这个子表达式的开始位置和结束位置. 这样我们就可以很容易把求值函数的框架写出来了:

```
eval(p, q) {
    if (p > q) {
        /* Bad expression */
    }
    else if (p == q) {
        /* Single token.
         * For now this token should be a number.
         * Return the value of the number.
         */
    }
    else if (check_parentheses(p, q) == true) {
        /* The expression is surrounded by a matched pair of parentheses.
         * If that is the case, just throw away the parentheses.
         */
        return eval(p + 1, q - 1);
    }
    else {
        /* We should do more things here. */
    }
}
```

其中 check_parentheses() 函数用于判断表达式是否被一对匹配的括号包围着, 同时检查表达式的左右括号是否匹配, 如果不匹配, 这个表达式肯定是不符合语法的, 也就不需要继续进行求值了. 我们举一些例子来说明 check_parentheses() 函数的功能:

```

"(2 - 1)"           // true
"(4 + 3 * (2 - 1))" // true
"4 + 3 * (2 - 1)"    // false, the whole expression is not surrounded by a matched pair of parentheses
"(4 + 3)) * ((2 - 1)" // false, bad expression
"(4 + 3) * (2 - 1)"  // false, the leftmost '(' and the rightmost ')' are not matched

```

至于怎么检查左右括号是否匹配,就留给聪明的你来思考吧!

上面的框架已经考虑了 BNF 中算术表达式的开头两种定义,接下来我们来考虑剩下的情况(即上述伪代码中最后一个 `else` 中的内容). 一个问题是, 给出一个最左边和最右边不同时是括号的长表达式, 我们要怎么正确地将它分裂成两个子表达式? 我们定义 **dominant operator** 为表达式人工求值时, 最后一步进行运行的运算符, 它指示了表达式的类型(例如当最后一步是减法运算时, 表达式本质上是一个减法表达式). 要正确地对一个长表达式进行分裂, 就是要找到它的 **dominant operator**.

我们继续使用上面的例子来探讨这个问题:

```

"4 + 3 * ( 2 - 1 )"
/*****/
case 1:
    "+"
    /  \
"4"    "3 * ( 2 - 1 )"

case 2:
    "*"
    /  \
"4 + 3" "( 2 - 1 )"

case 3:
    "-"
    /  \
"4 + 3 * ( 2" "1 )"

```

上面列出了 3 种可能的分裂, 注意到我们不可能在非运算符的 **token** 处进行分裂, 否则分裂得到的结果均不是合法的表达式. 根据 **dominant operator** 的定义, 我们很容易发现, 只有第一种分裂才是正确的. 这其实也符合我们人工求值的过程: 先算 4 和 $3*(2-1)$, 最后把它们的结果相加. 第二种分裂违反了算术运算的优先级, 它会导致加法比乘法更早进行. 第三种分裂破坏了括号的平衡, 分裂得到的结果均不是合法的表达式.

通过上面这个简单的例子, 我们就可以总结出如何在一个 **token** 表达式中寻找 **dominant operator** 了:

- 非运算符的 **token** 不是 **dominant operator**.
- 出现在一对括号中的 **token** 不是 **dominant operator**. 注意到这里不会出现有括号包围整个表达式的情况, 因为这种情况已经在 `check_parentheses()` 相应的 `if` 块中被处理了.
- **dominant operator** 的优先级在表达式中是最低的. 这是因为 **dominant operator** 是最后一步才进行的运算符.
- 当有多个运算符的优先级都是最低时, 根据结合性, 最后被结合的运算符才是 **dominant operator**. 一个例子是 $1+2+3$, 它的 **dominant operator** 应该是右边的 `+`.

要找出 **dominant operator**, 只需要将 **token** 表达式全部扫描一遍, 就可以按照上述方法唯一确定 **dominant operator**.

找到了正确的 **dominant operator** 之后, 事情就变得很简单了: 先对分裂出来的两个子表达式进行递归求值, 然后再根据 **dominant operator** 的类型对两个子表达式的值进行运算即可. 于是完整的求值函数如下:

```
eval(p, q) {
    if (p > q) {
        /* Bad expression */
    }
    else if (p == q) {
        /* Single token.
         * For now this token should be a number.
         * Return the value of the number.
         */
    }
    else if (check_parentheses(p, q) == true) {
        /* The expression is surrounded by a matched pair of parentheses.
         * If that is the case, just throw away the parentheses.
         */
        return eval(p + 1, q - 1);
    }
    else {
        op = the position of dominant operator in the token expression;
        val1 = eval(p, op - 1);
        val2 = eval(op + 1, q);
```

```
        switch (op_type) {
            case '+': return val1 + val2;
            case '-': /* ... */
            case '*': /* ... */
            case '/': /* ... */
            default: assert(0);
        }
    }
}
```

实现算术表达式的递归求值

由于 ICS 不是算法课, 我们已经把递归求值的思路和框架都列出来了. 你需要做的是理解这一思路, 然后在框架中填充相应内容. 实现表达式求值的功能之后, `p` 命令也就不难实现了. 需要注意的是, 上述框架中并没有进行错误处理, 在求值过程中发现表达式不合法的时候, 应该给上层函数返回一个表示出错的标识, 告诉上层函数“求值的结果是无效的”. 例如在 `check_parentheses()` 函数中, $(4+3)*(2-1)$ 和 $(4+3)*(2-1)$ 这两个表达式虽然都返回 `false`, 因为前一种情况是表达式不合法, 是没有办法成功进行求值的; 而后一种情况是一个合法的表达式, 是可以成功求值的, 只不过它的形式不属于 BNF 中的 $(\langle \text{expr} \rangle)$, 需要使用 `dominant operator` 的方式进行处理, 因此你还需要想办法把它们区别开来.

当然, 你也可以在发现非法表达式的时候使用 `assert(0)` 终止程序. 不过这样的话, 你在使用表达式求值功能的时候就要十分谨慎了.

实现带有负数的算术表达式的求值

在上述实现中, 我们并没有考虑负数的问题, 例如 `"1+-1"`, `"--1"` (我们不实现自减运算, 这里应该解释成 $-(-1)=1$), 它们会被判定为不合法的表达式. 为了实现负数的功能, 你需要考虑两个问题:

- 负号和减号都是 -, 如何区分它们?
- 负号是个单目运算符, 分裂的时候需要注意什么?

你可以选择不实现负数的功能, 但你很快就要面临类似的问题了.

调试中的表达式求值

实现了算术表达式的求值之后, 你可以很容易把功能扩展到复杂的表达式. 我们用 BNF 来说明需要扩展哪些功能:

```

<expr> ::= <decimal-number>
| <hexadecimal-number>      # 以"0x"开头
| <reg_name>                 # 以"$"开头
| "(" <expr> ")"
| <expr> "+" <expr>
| <expr> "-" <expr>
| <expr> "*" <expr>
| <expr> "/" <expr>
| <expr> "==" <expr>
| <expr> "!=" <expr>
| <expr> "&&" <expr>
| <expr> "||" <expr>
| "!" <expr>
| "*" <expr>                # 指针解引用

```

它们的功能和 C 语言中运算符的功能是一致的, 包括优先级和结合性, 如有疑问, 请查阅相关资料. 需要注意的是指针解引用(dereference)的识别, 在进行词法分析的时候, 我们其实没有办法把乘法和指针解引用区别开来, 因为它们都是*. 在进行递归求值之前, 我们需要将它们区别开来, 否则如果将指针解引用当成乘法来处理的话, 求值过程将会认为表达式不合法. 其实要区别它们也不难, 给你一个表达式, 你也能将它们区别开来. 实际上, 我们只要看*前一个 token 的类型, 我们就可以决定这个*是乘法还是指针解引用了, 不信你试试? 我们在这里给出 `expr()` 函数的框架:

```

if (!make_token(e)) {
    *success = false;
    return 0;
}

/* TODO: Implement code to evaluate the expression. */

for (i = 0; i < nr_token; i++) {
    if (tokens[i].type == '*' && (i == 0 || tokens[i - 1].type == certain type)) {
        tokens[i].type = Deref;
    }
}

return eval(?, ?);

```

其中的 `certain type` 就由你自己来思考啦! 其实上述框架也可以处理负数问题, 如果你之前实现了负数, * 的识别对你来说应该没什么困难了.

另外和 GDB 中的表达式相比, 我们做了简化, 简易调试器中的表达式没有类型之分, 因此我们需要额外说明两点:

- ❖ 为了方便统一, 我们认为所有结果都是 `uint32_t` 类型.
- ❖ 指针也没有类型, 进行指针解引用的时候, 我们总是从内存中取出一个 `uint32_t` 类型的整数, 同时记得使用 `vaddr_read()` 来读取内存.

实现更复杂的表达式求值

你需要实现上文 BNF 中列出的功能. 一个要注意的地方是词法分析中编写规则的顺序, 不正确的顺序会导致一个运算符被识别成两部分, 例如 `!=` 被识别成 `!` 和 `=`. 关于变量的功能, 它需要涉及符号表和字符串表的查找, 我们在 PA 中暂不实现.

上面的 BNF 并没有列出 C 语言中所有的运算符, 例如各种位运算, `<=` 等等. `==`, `!=` 和逻辑运算符很可能在使用监视点的时候用到, 因此要求你实现它们. 如果你在将来的使用中发现由于缺少某一个运算符而感到使用不方便, 到时候你再考虑实现它.

从表达式求值窥探编译器

你在程序设计课上已经知道, 编译是一个将高级语言转换成机器语言的过程. 但你是否曾经想过, 机器是怎么读懂你的代码的? 回想你实现表达式求值的过程, 你是否有什么新的体会?

事实上, 词法分析也是编译器编译源代码的第一个步骤, 编译器也需要从你的源代码中识别出 `token`, 这个功能也可以通过正则表达式来完成, 只不过 `token` 的类型更多, 更复杂而已. 这也解释了你为什么可以在源代码中插入任意数量的空白字符(包括空格, `tab`, 换行), 而不会影响程序的语义; 你也可以将所有源代码写在一行里面, 编译仍然能够通过.

一个和词法分析相关的有趣的应用是语法高亮. 在程序设计课上, 你可能完全没有想过可以自己写一个语法高亮的程序. 事实是, 这些看似这么神奇的东西, 其实也没那么复杂, 你现在确实有能力来实现它: 把源代码看作一个字符串输入到语法高亮程序中, 在循环中识别出一个 token 之后, 根据 token 类型用不同的颜色将它的内容重新输出一遍就可以了. 如果你打算将高亮的代码输出到终端里, 你可以使用 [ANSI 转义码的颜色功能](#).

在表达式求值的递归求值过程中, 逻辑上其实做了两件事情: 第一件事是根据 token 来分析表达式的结构 (属于 BNF 中的哪一种情况), 第二件事才是求值. 它们在编译器中也有对应的过程: 语法分析就好比分析表达式的结构, 只不过编译器分析的是程序的结构, 例如哪些是函数, 哪些是语句等等. 当然程序的结构要比表达式的结构更复杂, 因此编译器一般会使用一种标准的框架来分析程序的结构, 理解这种框架需要更多的知识, 这里就不展开叙述了. 另外如果你有兴趣, 可以看看 C 语言语法的 BNF.

和表达式最后的求值相对的, 在编译器中就是代码生成. ICS 理论课会有专门的章节来讲解 C 代码和汇编指令的关系, 即使你不了解代码具体是怎么生成的, 你仍然可以理解它们之间的关系. 这是因为 C 代码天生就和汇编代码有密切的联系, 高水平 C 程序员的思维甚至可以在 C 代码和汇编代码之间相互转换. 如果要深究代码生成的过程, 你也不难猜到是用递归实现的: 例如要生成一个函数的代码, 就先生成其中每一条语句的代码, 然后通过某种方式将它们连接起来.

我们通过表达式求值的实现来窥探编译器的组成, 是为了落实一个道理: 学习汽车制造专业不仅仅是为了学习开汽车, 是要学习发动机怎么设计. 我们也强烈推荐你在将来修读"编译原理"课程, 深入学习"如何设计发动机".

温馨提示

PA1 阶段 2 到此结束.

监视点

监视点的功能是监视一个表达式的值何时发生变化. 如果你从来没有使用过监视点, 请在 GDB 中体验一下它的作用. 简易调试器允许用户同时设置多个监视点, 删除监视点, 因此我们最好使用链表将监视点的信息组织起来. 框架代码中已经定义了监视点的结构.

在 `nemu/include/monitor/watchpoint.h` 中:

```
typedef struct watchpoint {
    int NO;
    struct watchpoint *next;

    /* TODO: Add more members if necessary */

} WP;
```

但结构体中只定义了两个成员: NO 表示监视点的序号, next 就不用多说了吧. 为了实现监视点的功能, 你需要根据你对监视点工作原理的理解在结构体中增加必要的成员. 同时我们使用"池"的数据结构来管理监视点结构体, 框架代码中已经给出了一部分相关的代码

(在 `nemu/src/monitor/debug/watchpoint.c` 中):

```
static WP wp_pool[NR_WP];
static WP *head, *free_;
```

代码中定义了监视点结构的池 `wp_pool`, 还有两个链表 `head` 和 `free_`, 其中 `head` 用于组织使用中的监视点结构, `free_` 用于组织空闲的监视点结构, `init_wp_pool()` 函数会对两个链表进行了初始化.

实现监视点池的管理

为了使用监视点池, 你需要编写以下两个函数 (你可以根据你的需要修改函数的参数和返回值):

```
WP* new_wp();
void free_wp(WP *wp);
```

其中 `new_wp()` 从 `free_` 链表中返回一个空闲的监视点结构, `free_wp()` 将 `wp` 归还到 `free_` 链表中, 这两个函数会作为监视点池的接口被其它函数调用. 需要注意的是, 调用 `new_wp()` 时可能会出现没有空闲监视点结构的情况,

为了简单起见,此时可以通过 `assert(0)` 马上终止程序. 框架代码中定义了 32 个监视点结构,一般情况下应该足够使用,如果你需要更多的监视点结构,你可以修改 `NR_WP` 宏的值. 这两个函数里面都需要执行一些链表插入,删除的操作,对链表操作不熟悉的同学来说,这可以作为一次链表的练习.

温故而知新

框架代码中定义 `wp_pool` 等变量的时候使用了关键字 `static`, `static` 在此处的含义是什么? 为什么要在此处使用它?

实现了监视点池的管理之后,我们就可以考虑如何实现监视点的相关功能了. 具体的,你需要实现以下功能:

- ✧ 当用户给出一个待监视表达式时,你需要通过 `new_wp()` 申请一个空闲的监视点结构,并将表达式记录下来. 每当 `cpu_exec()` 执行完一条指令,就对所有待监视的表达式进行求值(你之前已经实现了表达式求值的功能了),比较它们的值有没有发生变化,若发生了变化,程序就因触发了监视点而暂停下来,你需要将 `nemu_state` 变量设置为 `NEMU_STOP` 来达到暂停的效果. 最后输出一句话提示用户触发了监视点,并返回到 `ui_mainloop()` 循环中等待用户的命令.
- ✧ 使用 `info w` 命令来打印使用中的监视点信息,至于要打印什么,你可以参考 GDB 中 `info watchpoints` 的运行结果.
- ✧ 使用 `d` 命令来删除监视点,你只需要释放相应的监视点结构即可.

实现监视点

你需要实现上文描述的监视点相关功能,实现了表达式求值之后,监视点实现的重点就落在了链表操作上. 如果你仍然因为链表的实现而感到调试困难,请尝试学会使用 `assertion`. 在同一时刻触发两个以上的监视点也是有可能的,你可以自由决定如何处理这些特殊情况,我们对此不作硬性规定.

断点

断点的功能是让程序暂停下来,从而方便查看程序某一时刻的状态. 事实上,我们可以很容易地用监视点来模拟断点的功能: `w $eip==ADDR`

其中 `ADDR` 为设置断点的地址. 这样程序执行到 `ADDR` 的位置时就会暂停下来.

调试器设置断点的工作方式和上述通过监视点来模拟断点的方法大相径庭. 事实上,断点的工作原理,竟然是三十六计之中的"偷龙转凤"!如果你想揭开这一神秘的面纱,你可以阅读这篇文章. 了解断点的工作原理之后,可以尝试思考下面的两个问题.

一点也不能长?

我们知道 `int3` 指令不带任何操作数,操作码为 1 个字节,因此指令的长度是 1 个字节. 这是必须的吗? 假设有一种 x86 体系结构的变种 `my-x86`,除了 `int3` 指令的长度变成了 2 个字节之外,其余指令和 x86 相同. 在 `my-x86` 中,文章中的断点机制还可以正常工作吗? 为什么?

随心所欲"的断点

如果把断点设置在指令的非首字节(中间或末尾),会发生什么? 你可以在 GDB 中尝试一下,然后思考并解释其中的缘由.

NEMU 的前世今生

你已经对 NEMU 的工作方式有所了解了. 事实上在 NEMU 诞生之前, NEMU 曾经有一段时间并不叫 NEMU,而是叫 NDB (NJU Debugger), 后来由于某种原因才改名为 NEMU. 如果你想知道这一段史前的秘密,你首先需要了解这样一个问题: 模拟器 (Emulator) 和调试器 (Debugger) 有什么不同? 更具体地,和 NEMU 相比, GDB 到底是如何调试程序的?

i386 手册

在以后的 PA 中,你需要反复阅读 i386 手册. 鉴于有同学片面地认为"看手册"就是"把手册全看一遍",因而觉得"不可能在短时间内看完",我们在 PA1 的最后来聊聊如何科学地看手册.

学会使用目录

了解一本书都有哪些内容的最快方法就是查看目录,尤其是当你第一次看一本新书的时候. 查看目录之后并不代表你知道它们具体在说什么,但你会对这些内容有一个初步的印象,提到某一个概念的时候,你可以大概知道这个概念会在手册中的哪些章节出现. 这对查阅手册来说是极其重要的,因为我们每次查阅手册的时候总是关

注某一个问题,如果每次都需要把手册从头到尾都看一遍才能确定关注的问题在哪里,效率是十分低下的.事实上也没有人会这么做,阅读目录的重要性可见一斑.纸上得来终觉浅,还是来动手体会一下吧!

尝试通过目录定位关注的问题

假设你现在需要了解一个叫 `selector` 的概念,请通过 i386 手册的目录确定你需要阅读手册中的哪些地方.

逐步细化搜索范围

有时候你关注的问题不一定直接能在目录里面找到,例如"`CRO` 寄存器的 `PG` 位的含义是什么".这种细节的问题一般都是出现在正文中,而不会直接出现在目录中,因此你就不能直接通过目录来定位相应的内容了.根据你是否第一次接触 `CRO`,查阅这个问题会有不同的方法:

- ❖ 如果你已经知道 `CRO` 是个 `control register`,你可以直接在目录里面查看"`control register`"所在的章节,然后在这些章节的正文中寻找"`CRO`".
- ❖ 如果你对 `CRO` 一无所知,你可以使用阅读器中的搜索功能,搜索"`CRO`",还是可以很快地找到"`CRO`"的相关内容.不过最好的方法是首先使用搜索引擎,你可以马上知道"`CRO` 是个 `control register`",然后就可以像第一种方法那样查阅手册了.

不过有时候,你会发现一个概念在手册中的多个地方都有提到.这时你需要明确你要关心概念的哪个方面,通常一个概念的某个方面只会手册中的一个地方进行详细的介绍.你需要在这多个地方中进行进一步的筛选,但至少你已经过滤掉很多与这个概念无关的章节了.筛选也是有策略的,你不需要把多个地方的所有内容全部阅读一遍才能进行筛选,小标题,每段的第一句话,图表的注解,这些都可以帮助你很快地了解这一部分的内容大概在讲什么.这不就是高中英语考试中的快速阅读吗?对的,就是这样.如果你觉得目前还缺乏这方面的能力,现在锻炼的好机会来了.

搜索和筛选信息是一个 `trail and error` 的过程,没有什么方法能够指导你在第一遍搜索就能成功,但还是有经验可言的.搜索失败的时候,你应该尝试使用不同的关键字重新搜索.至于怎么变换关键字,就要看你对问题核心的理解了,换句话说,怎么问才算是切中要害.这不就是高中语文强调的表达能力吗?对的,就是这样.

事实上,你只需要具备一些基本的交际能力,就能学会查阅资料,和资料的内容没有关系,来一本"民法大全",`"XX 手机使用说明书"`,`"YY 公司人员管理记录"`,照样是这么查阅."`查阅资料`"是一种与领域无关的基本能力,无论身处哪一个行业都需要具备,如果你不想以后工作的时候被查阅资料的能力影响了自己的前途,从现在开始就努力锻炼吧!

必答题

你需要在实验报告中回答下列问题:

①查阅 i386 手册理解了科学查阅手册的方法之后,请你尝试在 i386 手册中查阅以下问题所在的位置,把需要阅读的范围写到你的实验报告里面:

- ✧ `EFLAGS` 寄存器中的 `CF` 位是什么意思?
- ✧ `ModR/M` 字节是什么?
- ✧ `mov` 指令的具体格式是怎么样的?

②`shell` 命令完成 `PA1` 的内容之后,`nemu/`目录下的所有 `.c` 和 `.h` 文件总共有多少行代码?你是使用什么命令得到这个结果的?和框架代码相比,你在 `PA1` 中编写了多少行代码?(Hint:目前 2017 分支中记录的正好是做 `PA1` 之前的状态,思考一下应该如何回到"过去"?)你可以把这条命令写入 `Makefile` 中,随着实验进度的推进,你可以很方便地统计工程的代码行数,例如敲入 `make count` 就会自动运行统计代码行数的命令.再来个难一点的,除去空行之外,`nemu/`目录下的所有 `.c` 和 `.h` 文件总共有多少行代码?

③使用 `man` 打开工程目录下的 `Makefile` 文件,你会在 `CFLAGS` 变量中看到 `gcc` 的一些编译选项.请解释 `gcc` 中的 `-Wall` 和 `-Werror` 有什么作用?为什么要使用 `-Wall` 和 `-Werror`?

温馨提示

`PA1` 到此结束。